

SWIPEX

Branch: AI & Machine Learning

Prepared by: Astha Sharma

Report Scope and Source Basis

This report follows the supplied SwipeX project specification for the project objective, modules, week-wise milestones, technology stack, evaluation criteria, performance metrics and deployment goals. Features that are part of the specification but are not confirmed in the current implementation are identified as planned/future functionality rather than being presented as completed.

Contents

1. Project Overview
2. Problem Statement
3. Proposed Solution
4. Objectives and Scope
5. System Users and Roles
6. System Architecture
7. Technology Stack
8. Milestone 1 – Requirements, Database Design & Core Setup
9. Milestone 2 – Swipe System & Job Discovery
10. Milestone 3 – Recommendations, Applications & Intelligent Features
11. Milestone 4 – Notifications, Analytics & Deployment
12. Key Functional Modules
13. Database Design
14. API & Backend Implementation
15. Frontend Implementation
16. Testing & Validation
17. Deployment Architecture
18. Challenges & Solutions
19. Performance Metrics & Evaluation
20. Future Scope
21. Conclusion

1. Project Overview

SwipeX is a swipe-based intelligent job discovery and career assistance platform. The project specification combines job discovery, recommendations, resume/ATS assistance, application tracking, notifications, competition tracking, dashboards and recruiter workflows in a centralized application.

The current project build uses a React frontend, Django REST backend, PostgreSQL production database, JWT authentication, job/application/saved-job workflows, recruiter features, notifications, recommendations, dashboard statistics and cloud deployment.

2. Problem Statement

- Job seekers often search through a large number of job listings manually.
- Finding opportunities according to skills, location, experience, salary and preferences can be time-consuming.
- Candidates need a convenient way to save jobs, apply and track application progress.
- Recruiters need an organized workflow for posting jobs and managing applicants.
- A modern platform can make job discovery more interactive and personalized.

3. Proposed Solution

SwipeX proposes a swipe-based job discovery experience in which jobs are presented through dynamic cards. Users can interact with opportunities and use supporting features such as saving, applying, recommendations, notifications and application tracking. Recruiters receive tools for job posting, job management, applicant review and recruitment monitoring.

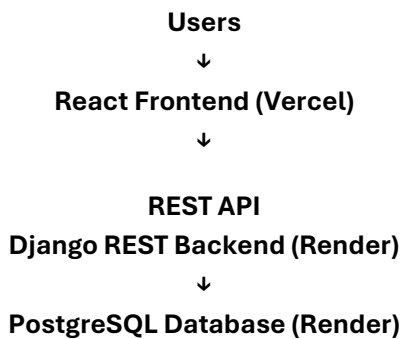
4. Objectives and Scope

- Build a swipe-based intelligent job discovery platform.
- Provide secure authentication and role-based access.
- Support job discovery, saving and application workflows.
- Provide personalized job recommendations.
- Track application status and recruitment activity.
- Provide notifications and competition-related alerts.
- Provide dashboards and analytics for application and hiring activity.
- Provide recruiter tools for job posting and applicant management.
- Deploy the platform to cloud environments with live URLs.

5. System Users and Roles

Role	Main Responsibilities
Job Seeker	Manage profile, discover jobs, save jobs, apply, track applications, view recommendations and notifications.
Recruiter	Manage company profile, post jobs, manage jobs, review applicants, manage hiring workflow and analytics.
Admin	The specification includes platform administration and platform-wide analytics; detailed admin implementation is not confirmed in the current build.

6. System Architecture:



The architecture separates the presentation layer, application/API layer and production database layer.

7. Technology Stack

Category	Technology / Tool
Frontend	React.js, React Router, Axios, Tailwind CSS
Backend	Python, Django, Django REST Framework
Database	PostgreSQL (production); SQLite listed for development
Authentication	JWT Authentication
Version Control	Git and GitHub
API Testing	Postman
Deployment	Vercel (frontend), Render (backend and PostgreSQL)
Containerization / DevOps	Docker and Docker Compose in the deployment plan
Development	VS Code

8. Milestone 1 – Requirements, Database Design & Core Setup

Timeline: Week 1 and Week 2.

- Defined project scope and user roles.
- Designed the database schema.
- Set up frontend and backend environments.
- Configured the database environment.
- Implemented authentication APIs.
- Built the initial frontend structure.

Outcome: frontend and backend architecture, authentication workflow and core database structure were established.

9. Milestone 2 – Swipe System & Job Discovery

Timeline: Week 3 and Week 4.

- Built job listing APIs.
- Developed dynamic job card interfaces.
- Implemented swipe-oriented job discovery.
- Implemented saving and application interactions.
- Developed job and company-related views.
- Worked toward filtering/search and recommendation-driven discovery.

Swipe Workflow

- Swipe Right: express interest / save or continue with the job workflow.
- Swipe Left: skip the job and continue discovery.
- Job cards display title, company, location, salary, job type and experience.

10. Milestone 3 – Recommendations, Applications & Intelligent Features

Timeline: Week 5 and Week 6.

The supplied specification includes resume upload/parsing, ATS scoring, resume-job compatibility and AI-generated suggestions. These are specification-level intelligent modules; they should not be described as completed unless demonstrated in the current build.

Implemented / Developed Workflows

- Application creation and application history.
- Application status tracking using Pending, Shortlisted, Interview and Rejected.
- Saved-job tracking.
- Personalized recommendation workflow based on saved-job interests.
- Applicant count and competition indicators.

Specification-Level AI Module

- Resume upload and parsing.
- ATS compatibility analysis.
- Keyword and missing-skill detection.
- Resume-job compatibility scoring.
- AI-generated resume improvement suggestions.
- Recommendation learning from swipe behavior.

11. Milestone 4 – Notifications, Analytics & Deployment

Timeline: Week 7 and Week 8.

Notifications

- User-specific notification storage and retrieval.
- Low-competition opportunity alerts based on applicant counts.
- Personalized recommendation notifications using saved-job interests.
- Notification types including recommendation and competition alerts.

Analytics & Dashboard

- Total job count.
- Applied-job and saved-job counts.
- Shortlisted, interview, rejected and pending application counts.
- Applicant count and competition information.

Deployment

- Django backend configured for production.
- Production PostgreSQL connected through environment variables.
- Django REST API deployed on Render.
- React frontend deployed on Vercel.
- Frontend connected to the live Render REST API.
- Git/GitHub used for source control and deployment updates.

12. Key Functional Modules

Module	Functionality
Authentication	Registration/login and JWT-protected APIs.
Job Discovery	Job listings and swipe-oriented discovery.
Saved Jobs	Save and retrieve jobs for authenticated users.
Applications	Apply to jobs and track application status.
Recommendations	Recommend jobs using saved-job interests.
Notifications	Low-competition and personalized recommendation alerts.
Recruiter Dashboard	Job and applicant management.
Job Posting	Create and manage job records.
Applicants	View applicants and manage application status.
Company Profile	Manage recruiter/company information.
Analytics	Dashboard counts and application-status statistics.

13. Database Design

Entity	Purpose
User	Authentication and user identity.
Company	Recruiter/company information.
Job	Title, description, company, location, salary, type, experience and posting time.
Application	Connects a user with a job and stores status and time.
SavedJob	Connects a user with a saved job.
Notification	Stores user-specific alerts, type, message, read state and creation time.

14. API & Backend Implementation

- JobViewSet provides CRUD-style job operations.
- SavedJobViewSet restricts saved jobs to the authenticated user.
- ApplyJobView creates an application for the authenticated user.
- AppliedJobsView lists applications.
- RecommendedJobsView returns recommended job records.
- DashboardStatsView calculates user-specific job and application statistics.
- NotificationView creates and returns notifications and generates recommendation/competition alerts.
- JWT authentication protects authenticated endpoints.

15. Frontend Implementation

- React Router provides navigation between job seeker and recruiter pages.
- Job seeker pages include dashboard, jobs, swipe jobs, saved jobs, applied jobs, recommendations, profile and notifications.
- Recruiter pages include recruiter dashboard, post job, my jobs, applicants, company profile and job editing.
- Axios is used for communication with the Django REST API.
- The production API URL connects the Vercel frontend with the Render backend.

16. Testing & Validation

Area	Validation
Authentication	Protected API behavior and JWT authorization were tested.
API	REST endpoints were tested during development and deployment.
Job Workflow	Job listing, saving and application workflows were connected to APIs.
Application Tracking	Application status categories and dashboard counts were implemented.
Notifications	Recommendation and low-competition notification generation was implemented.
Deployment	Render API returned successful API responses and Vercel communicated with the production backend.

17. Deployment Architecture

Layer	Platform	Purpose
Frontend	Vercel	Hosts the React production build and public frontend.
Backend	Render	Hosts the Django REST API.
Database	Render PostgreSQL	Stores production application data.
Source Control	GitHub	Stores source code and supports deployment workflow.

Environment variables are used for production database configuration so production connection details can be supplied by the cloud environment.

18. Challenges & Solutions

Challenge	Solution
Database migration changes	Updated Django migrations and applied them before deployment.
Local vs production database	Configured PostgreSQL through environment variables and treated production data separately.
Frontend initially using localhost	Updated Axios configuration to the Render production API URL.
401 Unauthorized API response	Added JWT authorization through an Axios request interceptor and re-authenticated.
400 error during job posting	Investigated the request payload and company relationship; a valid production company record is required.
Cloud deployment failure	Used Render logs to identify deployment dependency/configuration problems and redeployed.

19. Performance Metrics & Evaluation

- Recommendation accuracy and job matching relevance.
- ATS scoring effectiveness and resume analysis quality.
- Resume parsing, skill extraction and keyword detection accuracy.
- Dashboard response time and report generation performance.
- Recommendation processing efficiency.
- API response time.
- Concurrent user handling.
- Database query optimization.

The supplied specification identifies these metrics but does not provide measured numerical results; therefore no unsupported performance values are reported here.

20. Future Scope

- Complete and strengthen AI resume parsing and ATS scoring.
- Implement resume-job compatibility scoring.
- Add advanced skill-gap analysis.
- Improve recommendation learning from swipe behavior.
- Expand high-match and real-time notifications.
- Add richer hiring trend visualizations.
- Automate deployment with Docker Compose and GitHub Actions.
- Add stronger production monitoring, security and performance optimization.
- Extend the platform to mobile and additional career-assistance features.

21. Conclusion

SwipeX brings job discovery, application management, recruiter workflows, recommendations, notifications, dashboards and cloud deployment into one platform. The project progressed from requirements and core setup to swipe-based discovery, application and recommendation workflows, notifications and analytics, and finally cloud deployment.

The resulting architecture uses React.js for the frontend, Django REST Framework for backend APIs, PostgreSQL for production data storage, and Vercel/Render for cloud hosting. The supplied specification also provides a roadmap for deeper AI capabilities such as resume analysis, ATS scoring and skill-gap analysis.

References

- SwipeX project specification: “Swipe-Based Intelligent Job Discovery and Career Assistance Platform”.
- Django and Django REST Framework documentation used during development.
- React.js documentation used for frontend development.
- PostgreSQL used for production database deployment.
- GitHub, Render and Vercel used for source control and cloud deployment.